

MIDTERM EXAM REVIEW

10-701: INTRODUCTION TO MACHINE LEARNING

Fall 2026

1 Decision Trees

1. Suppose you have a training dataset consisting of 3 boolean features X_1, X_2 , and X_3 where $X_i \in \{0, 1\}$. Furthermore, assume the label is defined as $Y = X_1 \vee X_2$ i.e., $Y = 1$ if $X_1 = 1$ or $X_2 = 1$ and $Y = 0$ otherwise. Suppose that your training dataset contains all of the 8 possible feature vectors:

X_1	X_2	X_3	Y
0	0	0	0
1	0	0	1
0	1	0	1
1	1	0	1
0	0	1	0
1	0	1	1
0	1	1	1
1	1	1	1

- (a) What is the training error rate of a depth-0 decision tree on this dataset?

$\frac{1}{4}$

- (b) If your splitting criterion is training error rate, which feature or features would you split on first? Briefly justify your answer.

All features are equivalently good. No matter which feature you put at the root, the tree will still make 2 mistakes: 4 data points will go left and 4 data points will go right. Both sides will have more 1's than 0's, so the best tree will label both sides as positive (producing the same function as the depth-0 decision tree)

- (c) If your splitting criterion is information gain, which feature or features would you split on first? Briefly justify your answer.

Either X_1 or X_2 : X_3 has zero mutual information while both X_1 and X_2 have mutual information equal to $(-\frac{1}{4} \log \frac{1}{4} - \frac{3}{4} \log \frac{3}{4}) - \frac{1}{2} > 0$

2. You are given a dataset for binary classification with two features x_1 and x_2 . x_1 can have two possible values 0,1 and x_2 can have three possible values 0,1,2. Figure 1 provides a depiction of this dataset.

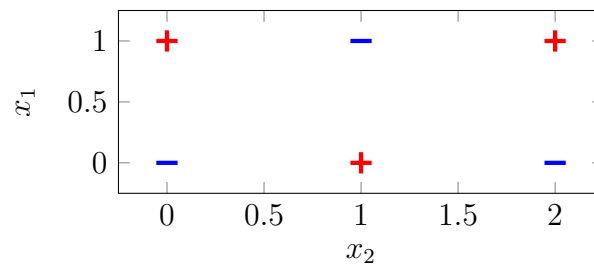


Figure 1: Binary classification dataset.

- (a) What is the lowest possible training error rate any decision tree could achieve on this dataset? The tree need not be a binary tree but each node should split on at most one feature.

- (b) Draw a decision tree that achieves this training error rate.

Many possible solutions exist but the most intuitive one first split on $x_1 < 0.5$ and then makes a ternary split on x_2 down either branch at 0.5 and 1.5.

3. We would like to learn a decision tree. We have n samples for training. You can assume that n is large and we are using continuous features. In the following questions, suppose we only use the first feature, x_1 .

(a) We would like to split according to x_1 at the root with 3 branches: data points are split at the root to three different subtrees by findings values a and b such that the three subtrees are $x_1 \leq a$, $a < x_1 < b$ and $x_1 \geq b$. What is the runtime for finding the values of x_1 that should be used by the root for such a split?

- ☐ $O(\log n)$
- ☐ $O(\sqrt{n})$
- ☐ $O(n)$
- ☐ $O(n^2)$
- ☐ $O(n^3)$

D. To split on continuous values we need to find the best thresholds a and b . This requires us to order the values of x_1 and then test all possible splits and so the runtime is $O(n^2)$.

(b) Following our first split, we would like to *split again* on x_1 in each of the three sub-trees resulting from the split in the previous question. Again, for each subtree we would split three ways as we did in the root. What is the total runtime for determining the optimal splits for ALL three subtrees?

- ☐ $O(n)$
- ☐ $O(n^2)$
- ☐ $O(n^3)$
- ☐ $O(n^4)$
- ☐ $O(n^8)$

B. Let the number of samples assigned to each of the three subtrees be n_1 , n_2 and n_3 . We know from question 1 that the total run time for each is $O(n_1^2)$, $O(n_2^2)$, $O(n_3^2)$. Since $n_1 + n_2 + n_3 = n$ one of these sets is $O(n)$ and so the total runtime is $O(n^2)$.

(c) For the same dataset we would like to learn the *optimal* tree that:

- Only uses x_1
- Has two levels, a root and a level below it where each splits to three branches (see the figure below).



What is the runtime for computing the *optimal tree*, in terms of minimizing the training error rate?

- ☐ $O(n)$
- ☐ $O(n^2)$
- ☐ $O(n^4)$
- ☐ $O(n^8)$
- ☐ $O(n^9)$

C. A naive solution would be to simply find the best 9 way split, which would entail a total run time is $O(n^8)$. However, we can formulate the problem as a dynamic program where we enumerate all $O(n^2)$ possible splits at the top level, then brute force over the $O(n^2)$ possible splits down each of the subtrees.

Similarly, we can assess each split in $O(1)$ time by caching statistics. For example, to assess training accuracy we need the count of positives and negatives to the left and right of the split, so as we pass over each example we subtract it from the right and add it to the left.

2 kNN

1. **True or False:** Regardless of the training dataset size, the true error rate of a 1-NN model will never exceed 2 * the true error rate of the optimal classifier on a binary classification problem with stochastic labels.

☐ True

☐ False

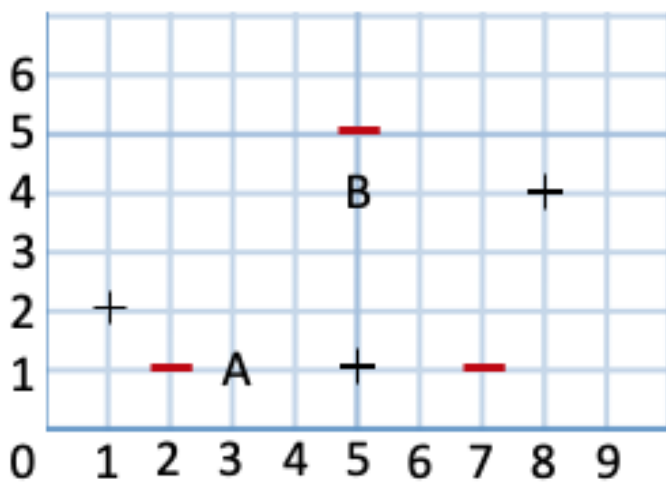
False, this condition only holds with a high probability as $N \rightarrow \infty$

2. Suppose you are deciding between the following k NN models for a binary classification problem where $y \in \{-1, +1\}$ (for both models, assume we use the Euclidean distance):
 - Model 1: Probabilistic 3NN. This model predicts the label of a query data point with probabilities corresponding to the relative proportion of labels in the 3 nearest neighbors, e.g., if the 3 nearest neighbors have labels $\{-1, -1, +1\}$ then this model predicts -1 with probability $2/3$ and $+1$ with probability $1/3$.
 - Model 2: Distance weighted 2NN. This model predicts

$$\hat{y} = \text{sign} \left(\sum_{(\vec{x}^{(i)}, y^{(i)}) \in N_2(\vec{x})} \frac{y^{(i)}}{\|\vec{x}^{(i)} - \vec{x}\|_2^2 + 1} \right)$$

where $N_2(\vec{x})$ are the 2 nearest neighbors in the training data to the query data point $\vec{x}$ (the plus one in the denominator is to avoid division by 0). For the purposes of this model, let $\text{sign}(0) = +1$.

You gather a training dataset of 6 points: 3 red $-$'s, which correspond to label $y = -1$, and 3 black $+$'s, which correspond to label $y = +1$. You also gather a validation dataset consisting of points A and B, both of which have label $+1$.



- (a) What is the *expected* training error rate for Model 1? You may express your answer as a fraction if necessary.

Starting with the -1 at $(5,5)$ and going clockwise, the expected pointwise training errors are

$$\frac{2}{3} + \frac{2}{3} + \frac{2}{3} + \frac{2}{3} + \frac{2}{3} + \frac{1}{3} = \frac{11}{3}$$

Thus, the expected training error rate is $\frac{11}{18}$.

- (b) What is the training error rate for Model 2? You may express your answer as a fraction if necessary.

0; the trick here is to realize that this is effectively a 1NN model when applied to the training data as the nearest neighbor to a training data point will always be itself and thus, will always have a higher weight than the second nearest neighbor.

- (c) What is the *expected* validation error rate for Model 1? You may express your answer as a fraction if necessary.

$\frac{1}{3}$; the expected validation error on both A and B is $\frac{1}{3}$ as the 3 nearest neighbors to A and B both have label sets $\{-1, +1, +1\}$.

- (d) What is the validation error rate for Model 2? You may express your answer as a fraction if necessary.

1; again, because no two points are equidistant to either A or B, this model boils down to a 1NN.

3. For the points shown in Figure 2 suppose x_1, x_4, x_5, x_7 , and x_9 have label $+1$, and the other points have label -1 . For a 3NN classifier using the Euclidean distance, what is the LOOCV error?

The predictions are below. The error is 1

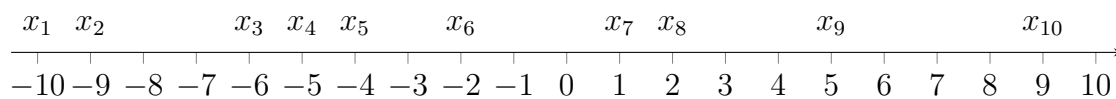


Figure 2

point	ground truth	LOOCV prediction
x_1	+1	-1
x_2	-1	+1
x_3	-1	+1
x_4	+1	-1
x_5	+1	-1
x_6	-1	+1
x_7	+1	-1
x_8	-1	+1
x_9	+1	-1
x_{10}	-1	+1

3 Linear Regression & Regularization

- Using monthly stock averages, $x_1, x_2, \dots, x_n$, you run linear regression without regularization to estimate some future stock value y . However, your test error rate turns out to be very high. Your friend suggests that maybe the price *differences* between consecutive months may offer better features, so you add additional features $(x_1 - x_2), (x_2 - x_3), (x_3 - x_4), \dots, (x_{n-1} - x_n)$. Do you expect an improvement? Briefly justify your answer.

No because the new features are just a linear combination of the original features. Thus, any regressor that could be learned using the new feature set would already have been considered by the original linear regression model.

- Consider the following linear regression model:

$$\mathbf{y} = \mathbf{X}\mathbf{w} + \boldsymbol{\epsilon}$$

where we assume the residuals are normal and i.i.d.: $\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \sigma^2 \mathbf{I})$. Recall from lecture that the MLE of $\mathbf{w}$ is $\hat{\mathbf{w}} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}$. Using the fact that $\hat{\mathbf{w}}$ is an unbiased estimator of $\mathbf{w}$, derive an expression for the variance of $\text{var}(\hat{\mathbf{w}})$:

By definition,

$$\text{var}(\hat{\mathbf{w}}) = \mathbb{E}[\hat{\mathbf{w}}^2] - (\mathbb{E}[\hat{\mathbf{w}}])^2$$

We can calculate $\mathbb{E}[\hat{\mathbf{w}}^2]$ as,

$$\begin{aligned} \mathbb{E}[\hat{\mathbf{w}}^2] &= \mathbb{E}[(\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}]^2 \\ &= \mathbb{E}[(\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T (\mathbf{X}\mathbf{w} + \boldsymbol{\epsilon})]^2 \\ &= \mathbb{E}[(\mathbf{w} + (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \boldsymbol{\epsilon})^2] \\ &= \mathbf{w}^2 + 2\mathbb{E}[(\mathbf{X}^T \mathbf{X})^{-1} \boldsymbol{\epsilon}] + (\mathbf{X}^T \mathbf{X})^{-1} \mathbb{E}[\boldsymbol{\epsilon}^2] \\ &= \mathbf{w}^2 + (\mathbf{X}^T \mathbf{X})^{-1} \mathbb{E}[\boldsymbol{\epsilon}^2] \end{aligned}$$

Because $\hat{\mathbf{w}}$ is unbiased, $\mathbb{E}[\hat{\mathbf{w}}] = \mathbf{w}$. Thus, $\text{var}(\hat{\mathbf{w}}) = \mathbf{w}^2 + (\mathbf{X}^T \mathbf{X})^{-1} \mathbb{E}[\boldsymbol{\epsilon}^2] - \mathbf{w}^2 = (\mathbf{X}^T \mathbf{X})^{-1} \sigma^2$.

4 MLE/MAP

1. **True or False:** Recall that the MLE of a parameter θ given some dataset $\mathcal{D}$ is

$$\hat{\theta}_{\text{MLE}} = \arg \max_{\theta} p(\mathcal{D}|\theta)$$

while the MAP estimate of θ is

$$\hat{\theta}_{\text{MAP}} = \arg \max_{\theta} p(\mathcal{D}|\theta)p(\theta).$$

For every dataset, there will always exist some prior distribution $p(\theta)$ for which the MAP estimate is equivalent to the MLE.

☐ True

☐ False

True; for bounded θ , the uniform distribution works, and for unbounded θ , any unimodal distribution with mode $\arg \max_{\theta} p(\mathcal{D}|\theta)$ works.

2. The probability density function of the Pareto distribution is

$$p(x|k, \alpha) = \begin{cases} \frac{\alpha k^\alpha}{x^{\alpha+1}} & x \in [k, \infty) \\ 0 & \text{otherwise} \end{cases}$$

$$k, \alpha \in (0, \infty)$$

Given n independent samples $x_1, x_2, \dots, x_n$ drawn from a Pareto distribution, what is the MLE for the parameters k and α ? **Hint: first determine the MLE of k , then use that result to in your expression for the MLE of α .**

$$\hat{k}_{MLE} = \underline{\hspace{2cm}}$$

$$\hat{\alpha}_{MLE} = \underline{\hspace{2cm}}$$

For $x_i \geq k$, likelihood is

$$L(k, \alpha) = \prod_{i=1}^n \frac{\alpha k^\alpha}{x_i^{\alpha+1}} = \frac{\alpha^n k^{n\alpha}}{(\prod_{i=1}^n x_i)^{\alpha+1}}$$

and log likelihood is $\ell(k, \alpha) = n \ln \alpha + n\alpha \ln k - (\alpha + 1) \sum_{i=1}^n \ln x_i$

$$\begin{aligned} \frac{n}{\alpha} + n \ln(k) - \sum_{i=1}^n \ln(x_i) &= \frac{n}{\alpha} + \sum_{i=1}^n (\ln(k) - \ln(x_i)) \\ &= \frac{n}{\alpha} - \sum_{i=1}^n (\ln(x_i) - \ln(k)) \\ &= \frac{n}{\alpha} - \sum_{i=1}^n \ln\left(\frac{x_i}{k}\right) \end{aligned}$$

$$\hat{k}_{MLE} = \min_i x_i$$

$$\hat{\alpha}_{MLE} = \frac{n}{\sum_{i=1}^n \ln\left(\frac{x_i}{\hat{k}_{MLE}}\right)}$$

5 Logistic Regression

1. Consider a dataset with only binary features, $\mathbf{x} \in \{0, 1\}^d$, where feature x_1 is rare and only takes on value 1 in the training dataset if the corresponding the label is 1. If you fit a logistic regression model to this dataset using gradient descent, what value will the coefficient on this feature, $\hat{w}_1$, approach? Briefly justify your answer.

If a binary feature fired for only label 1 in the training set then, by maximizing the conditional log likelihood, we will make the weight associated to that feature be infinite. This is because, when this feature is observed in the training set, we will want to predict predict 1 irrespective of everything else. This is an undesired behaviour from the point of view of generalization performance, as most likely we do not believe this rare feature to have that much information about class 1. Most likely, it is spurious co-occurrence. Controlling the norm of the weight vector will prevent these pathological cases.

2. A generalization of logistic regression to a multiclass settings involves expressing the per-class probabilities $P(y = c|x)$ as the softmax function $\frac{\exp(w_c^T x)}{\sum_{d \in C} \exp(w_d^T x)}$, where c is some class from the set of all classes C and w_c is a class-specific weight vector.

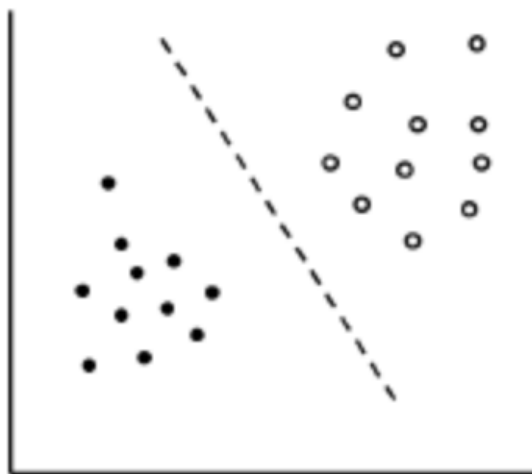
Consider a binary classification problem with labels encoded as $y \in \{0, 1\}$. Rewrite the expression above for this situation to end up with expressions for $P(Y = 1|x)$ and $P(Y = 0|x)$ that we derived in class for binary logistic regression.

$$P(y = 1|x) = \frac{\exp(w_1^T x)}{\exp(w_0^T x) + \exp(w_1^T x)} = \frac{\exp((w_1 - w_0)^T x)}{1 + \exp((w_1 - w_0)^T x)} = \frac{\exp(w^T x)}{1 + \exp(w^T x)} = p$$

Therefore, $1 - p = \frac{1}{1 + \exp(w^T x)}$

6 Regularization

- The plot below shows data from two classes (filled and unfilled circles) separated by a linear decision boundary (dashed) defined by a two-dimensional weight vector and a bias term.



Suppose we apply L_2 regularization *to the bias term only* i.e., the two-dimensional weight vector remains the same for the regularized and unregularized settings. Which of the following best describes the new decision boundary that would result from this change. Assume that the amount of regularization is mild.

- ☐ farther from the origin; parallel to the previous line
- ☐ farther from the origin; not parallel to the previous line
- ☐ closer to the origin; parallel to the previous line
- ☐ closer to the origin; not parallel to the previous line

C

- Complete the following paragraph about L_1 and L_2 regularization by circling the best of the provided options for each of blanks:

When applied to linear regression, L_1 regularization tends to learn

denser / sparser weight vectors than L_2 regularization because the

constraint surface is non-convex / square-shaped. The optimal

weight vector for linear regression with L_1 regularization can / cannot

be solved for in closed form.

When applied to linear regression, L_1 regularization tends to learn sparser weight vectors than L_2 regularization because the constraint surface is square-shaped. The optimal weight vector for linear regression with L_1 regularization cannot be solved for in closed form.

7 Neural Networks & Backpropagation

1. Your friend wants to build a fully-connected one-hidden-layer Neural Network. Her inputs have 9 features. She wants 7 hidden neurons in the first layer and 2 neurons in the output layer generated using a softmax function. Each layer is a linear layer fully-connected to the previous one and includes a bias term. Each scalar counts as one parameter.

- (a) What is the number of parameters she will need to create this neural network? Each scalar counts as one parameter.

(9+1)x7 for the first layer is 70, (7+1)x2 for the output layer is 16, so the total is 86.

- (b) Your friend shares with you that the output she is trying to predict is binary. Using this information, describe an architecture that can learn the same decision boundary while reducing the number of parameters.

She could make the output layer be only one node which contains the sigmoid function. Using a threshold of 0.5 will then recreate the predictions of the softmax.

2. In backpropagation, why is it necessary that we loop through the layers in reverse-order, moving from the output layer backwards towards the input layer?

The gradient of the loss with respect to the weights in some layer ℓ depends on the gradient with respect to all of the weights in downstream layers or layers closer to the output layer. Hence in order to compute these earlier gradients using the chain rule, backpropagation has to first compute the gradient with respect to weights in later layers.

3. When training a neural network, which of the following techniques is/are *primarily* used to address the nonconvexity of the loss function?

- ☐ Momentum
- ☐ Weight decay
- ☐ Random restarts
- ☐ Early termination
- ☐ None of the above

A and C